Semanas 9 y 10 · Paso a paso detallado

Versión ampliada de las guías de las semanas 9 y 10, con el último ajuste de firmware antes de congelar el entorno.

Antes de empezar: último cambio de firmware

La semana 9 **congela** el firmware y el código: desde ese momento no se puede cambiar nada hasta terminar la campaña. Por eso, antes de congelar, se corrige la medición de memoria.

> **Por qué hace falta**
>
> En la semana 7, clásico e híbrido dieron casi la misma memoria (10.976 y 10.968 bytes). El firmware medía la memoria que queda ocupada **después** del handshake, no el máximo que se usa **durante** el intercambio, que es donde ML-KEM necesita más. El nuevo `cliente_tls.c` registra cada reserva de memoria de wolfSSL y guarda el **pico**. El campo sigue llamándose `esp_heap_usado_hs`, pero ahora significa «memoria pico de wolfSSL desde la creación del contexto hasta el fin del handshake».

Paso 0.1. Reemplazar el archivo y grabar (laptop)

- Reemplaza `esp32\firmware\main\cliente_tls.c` por el archivo que te envié.
- Revisa que `esp32\firmware\main\CMakeLists.txt` tenga `set(FRAME_SIZE 128)` y `set(MODO_ECO 0)`.
- Con el USB del ESP32 en la laptop, en la terminal de ESP-IDF: `idf.py -p COM5 build`. Cuando termine sin errores, mantén presionado **BOOT**, ejecuta `idf.py -p COM5 flash` y suelta BOOT cuando aparezca `Connecting...`.
- Debe terminar con `Hash of data verified`. Regresa el USB a la Raspberry.

Paso 0.2. Comprobar la nueva medición (Raspberry)

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 servidor_tls.py --grupo clasico --frame 128 | grep -E 'esp_estado|heap_usado'
python3 servidor_tls.py --grupo hibrido --frame 128 | grep -E 'esp_estado|heap_usado'
```

Los dos con `"esp_estado": 0`. Ahora `esp_heap_usado_hs` debe ser **distinto** entre clásico e híbrido (se espera mayor en el híbrido). Anota los dos valores en la bitácora.

Paso 0.3. Actualizar el diccionario de datos y marcar el firmware final

En `docs/diccionario_datos.md` cambia la descripción de `esp_heap_usado_hs` por: *memoria dinámica pico reservada por wolfSSL desde la creación del contexto hasta el fin del handshake (firmware ≥ semana 9)*. Luego, en la Raspberry:

```
cd ~/proyecto-kyber-rpi
git add -A && git commit -m "Firmware final de campaña: memoria pico de wolfSSL"
git tag -a v0.9-campana -m "Firmware y código congelados para la campaña"
git push && git push origin v0.9-campana
```

Si el código del ESP32 está solo en la laptop, copia también ahí el `cliente_tls.c` nuevo al repositorio y súbelo, para que el commit refleje lo que está grabado.

> **El analizador lógico, fuera de la campaña**
>
> Durante las semanas 9 y 10 el analizador debe estar **desconectado** del bus: altera el enlace (más errores de CRC) y también el consumo. Las capturas de la semana 8 se hacen después, en una sesión aparte.

SEMANA 9 · Entorno congelado, piloto y tamaño de muestra

Objetivo	Congelar el entorno, hacer una prueba piloto en los dos modos del bus y calcular cuántas repeticiones necesita la campaña.
Dedicación estimada	4–6 horas (dos reinicios y dos pilotos de ~15 min).
Antes de empezar	Firmware final grabado y etiqueta v0.9-campana (pasos 0.1 a 0.3).
Al terminar tendrán	Pilotos estándar y rápido, tamano_muestra.csv y el número de repeticiones N decidido.

Paso 1. Congelar y documentar el entorno

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
bash ../herramientas/verificar_entorno.sh
systemctl list-units --type=service --state=running > ../docs/evidencias/servicios_campana.txt
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
git -C .. rev-parse --short HEAD
```

Abre `nano ../docs/decisiones.md`, ve al final y pega este bloque completando los datos:

```
## Semana 9 - Entorno de la campaña
- Fecha: ____
- Commit congelado: ____ (etiqueta v0.9-campana). FRAME_SIZE = 128.
- Gobernador de CPU: performance (se fija después de cada reinicio).
- Red: SSH por Wi-Fi; Wi-Fi y Bluetooth se mantienen igual en todas las mediciones.
- ESP32 alimentado por USB desde la Raspberry; analizador lógico desconectado.
- esp_heap_usado_hs = memoria pico de wolfSSL (definición nueva desde este commit).
```

Guarda con Ctrl+O, Enter y sal con Ctrl+X. Si entras a la Raspberry por cable Ethernet podrías desactivar Wi-Fi y Bluetooth; como entras por Wi-Fi, **no los desactives**.

Paso 2. Piloto en modo estándar (100 kbit/s)

```
sudo nano /boot/firmware/config.txt
#   deja la línea así:  dtoverlay=i2c3,pins_4_5,baudrate=100000
#   Ctrl+O, Enter, Ctrl+X
sudo reboot
```

Espera 1 minuto, vuelve a entrar por SSH y comprueba todo antes de medir:

```
echo performance | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
grep i2c3 /boot/firmware/config.txt
vcgencmd measure_temp
vcgencmd get_throttled
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 servidor_tls.py --grupo clasico --frame 128 | grep -E 'esp_estado|"error"'
```

Comprobación	Debe dar
governor	performance (cuatro veces)
grep i2c3	baudrate=100000
measure_temp	menos de 60 °C (si no, espera con la Raspberry en reposo)
get_throttled	throttled=0x0
servidor_tls.py	"esp_estado": 0 y sin línea "error" con contenido

Ahora el piloto, dentro de tmux (dura unos 15 minutos):

```
tmux new -s piloto
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 campana.py --bloque estandar --piloto --reps 10 --frame 128
```

- Son 60 corridas: 2 grupos × 3 tamaños de mensaje (32, 128 y 512 bytes) × 10, en orden aleatorio, con 60 s de reposo al inicio y al final para medir la potencia en reposo.
- Puedes salir de tmux sin detenerlo (**Ctrl+B**, suelta, **D**) y volver con `tmux attach -t piloto`. **No escribas nada en la Raspberry mientras mide**: teclear por SSH también consume energía.
- Cuando termine:

```
python3 resumen_rapido.py | tee ../docs/evidencias/s9_piloto_estandar.txt
```

Debe decir **Corridas: 60 correctas: 60** (o casi) y **Consistencia: OK**. Sal de tmux escribiendo `exit`.

Paso 3. Piloto en modo rápido (400 kbit/s)

```
sudo nano /boot/firmware/config.txt
#   deja la línea así:  dtoverlay=i2c3,pins_4_5,baudrate=400000
#   Ctrl+O, Enter, Ctrl+X
sudo reboot
```

Espera 1 minuto, vuelve a entrar por SSH y comprueba todo antes de medir:

```
echo performance | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
grep i2c3 /boot/firmware/config.txt
vcgencmd measure_temp
vcgencmd get_throttled
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 servidor_tls.py --grupo clasico --frame 128 | grep -E 'esp_estado|"error"'
```

Comprobación	Debe dar
governor	performance (cuatro veces)
grep i2c3	baudrate=400000
measure_temp	menos de 60 °C (si no, espera con la Raspberry en reposo)
get_throttled	throttled=0x0
servidor_tls.py	"esp_estado": 0 y sin línea "error" con contenido

```
tmux new -s piloto
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 campana.py --bloque rapido --piloto --reps 10 --frame 128
python3 resumen_rapido.py | tee ../docs/evidencias/s9_piloto_rapido.txt
```

> **Si un piloto sale mal**
>
> Si más de 3 corridas fallan, no sigas: mándame el resumen. Si repites un piloto, mueve el anterior a otra carpeta (`mkdir -p ../data/descartados && mv ../data/raw/piloto_rapido_* ../data/descartados/`) para que el cálculo del paso 4 no mezcle dos ejecuciones.

Paso 4. Calcular el tamaño de muestra

```
mkdir -p ../data/processed
python3 tamano_muestra.py ../data/raw/piloto_*_meta.json \
        | tee ../docs/evidencias/s9_tamano_muestra.txt
```

El script aplica la fórmula de la propuesta: n = (100·z·s / (r·media))², con precisión r = 5 % y 95 % de confianza (Jain, 1991), y un mínimo de 30 (Montgomery y Runger, 2014). Muestra tres cosas:

- Una tabla con el **n_requerido** para cada métrica (tiempo de handshake en el ESP32 y en la Raspberry, energía por handshake y rendimiento), por bloque y grupo.
- La línea **Repeticiones recomendadas por configuración: N**.
- Una tabla con la precisión (± %) que se lograría con 30, 60 y 100 repeticiones.

Además guarda todo en `data/processed/tamano_muestra.csv`.

Paso 5. Decidir N

Calcula cuánto tardaría **un bloque** de la campaña:

```
corridas por bloque = 6 × N
tiempo ≈ corridas × (duración media por corrida + 3 s de pausa) + 2 min de reposo
```

La duración media por corrida aparece en el resumen del piloto (por ejemplo, 7,2 s). Ejemplo con N = 30: 180 × (7,2 + 3) ≈ 1.836 s ≈ **31 min** + 2 min ≈ 33 min por bloque.

Si el script recomienda…	Qué hacer
un N que da hasta ~3 h por bloque	usa ese N
un N enorme (suele pasar con la energía)	fija un N factible (por ejemplo 60 o 100) y reporta la precisión lograda con la tabla del script
menos de 30	usa 30 (es el mínimo de la propuesta)

Anota la decisión en `docs/decisiones.md` con los números, por ejemplo: *«N = 60. El tiempo de handshake requiere n = 12 y la energía n = 240; con N = 60 la energía se estima con ±10 % de precisión (tabla de tamano_muestra.csv)»*.

Paso 6. Ensayo general del análisis

```
python3 analisis.py ../data/raw/piloto_*_meta.json
```

- Debe crear `data/processed/resumen.csv`, `comparaciones.csv`, `corridas_con_energia.csv` y figuras en `docs/figuras/`.
- En `corridas_con_energia.csv`, la columna `muestras_en_hs` indica cuántas lecturas del INA219 caen dentro de cada handshake. Más de 50 es cómodo.
- En `reposo_por_bloque.csv`, la diferencia de potencia de reposo entre el inicio y el final de cada bloque (deriva) debe ser pequeña.

Si algo falla aquí, avísame **antes** de la semana 10: es mejor corregir el análisis con los datos del piloto que con la campaña completa.

Paso 7. Commit

```
cd ~/proyecto-kyber-rpi
git add -A && git commit -m "Semana 9: pilotos, tamaño de muestra y N decidido"
git push
```

Criterio de cierre de la semana

- ☐ Firmware con memoria pico grabado y etiqueta v0.9-campana.
- ☐ Entorno documentado en decisiones.md (commit, gobernador, red, FRAME_SIZE).
- ☐ Pilotos estándar y rápido de 60 corridas, sin fallas importantes.
- ☐ tamano_muestra.csv generado y N decidido y justificado en decisiones.md.
- ☐ analisis.py funcionó con el piloto.
- ☐ Commit y push.

SEMANA 10 · Campaña de mediciones

Objetivo	Ejecutar la campaña completa en dos bloques (100 y 400 kbit/s) con el N decidido, en las mismas condiciones, y respaldar los datos.
Dedicación estimada	Según N (con N = 30, unos 35 min por bloque).
Antes de empezar	Semana 9 cerrada y N anotado en decisiones.md.
Al terminar tendrán	Dos bloques completos en data/raw/, respaldados en GitHub y en la laptop.

Paso 1. Sortear el orden de los bloques

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 -c "import random; b=['estandar','rapido']; random.Random(42).shuffle(b); print(b)"
```

Anota el orden que salga en `docs/decisiones.md` y respétalo. Dentro de cada bloque el orden de las corridas ya es aleatorio (semilla 42).

Paso 2. Preparar el primer bloque

Usa el baudrate del primer bloque del sorteo: **100000** si es «estandar» o **400000** si es «rapido». Reinicia y haz las mismas comprobaciones del piloto:

```
sudo nano /boot/firmware/config.txt      # baudrate del bloque
sudo reboot
#   ... después de volver a entrar:
echo performance | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
grep i2c3 /boot/firmware/config.txt
vcgencmd measure_temp; vcgencmd get_throttled
```

Lista de chequeo antes de CADA bloque

- ☐ baudrate del bloque en config.txt y Raspberry reiniciada.
- ☐ Gobernador en performance.
- ☐ Temperatura menor a 60 °C y throttled=0x0.
- ☐ ESP32 por USB a la Raspberry; miniterm cerrado; analizador desconectado.
- ☐ Ningún otro programa del proyecto corriendo.
- ☐ venv activado (OPENSSL_CONF) y --frame 128 igual al firmware.

Anota en la bitácora la fecha, la hora y la temperatura inicial.

Paso 3. Ejecutar el bloque

```
tmux new -s campana
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 campana.py --bloque estandar --reps N --frame 128
#   cambia «estandar» por «rapido» según el bloque, y N por el número decidido
```

- Sal de tmux con **Ctrl+B** y luego **D**; puedes cerrar el SSH y la laptop. La campaña sigue.
- Para ver cómo va: `tmux attach -t campana`.
- El script deja 60 s de reposo al inicio y al final, pausa 3 s entre corridas y espera solo si la temperatura pasa de 75 °C.
- **No uses la Raspberry mientras mide** (ni comandos ni transferencias).

Paso 4. Revisar y respaldar al terminar el bloque

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 resumen_rapido.py | tee ../docs/evidencias/s10_resumen_BLOQUE.txt
cd ~/proyecto-kyber-rpi/data/raw && gzip -k campana_*_potencia.csv
cd ~/proyecto-kyber-rpi
git add data/raw/*.gz data/raw/*_corridas.jsonl data/raw/*_meta.json docs/
git commit -m "Campaña: bloque BLOQUE terminado" && git push
```

Cambia **BLOQUE** por estandar o rapido. Luego, en una PowerShell **de la laptop**, copia los datos a una carpeta de respaldo:

```
mkdir $HOME\Documents\campana -Force; cd $HOME\Documents\campana
scp "usuariok@integradoraPi4.local:~/proyecto-kyber-rpi/data/raw/campana_*" .
```

Cuándo repetir un bloque

Las corridas con error, con esp_estado distinto de 0 o con throttling se excluyen solas en el análisis y se reportan. Si en un bloque falla **más del 5 %** (con N = 30, más de 9 de 180), o la potencia de reposo cambia mucho entre el inicio y el final, busca la causa, anótala, corrígela y repite **el bloque completo**. Mueve el bloque fallido a `data/descartados/`: no se mezclan corridas de dos ejecuciones del mismo bloque.

Paso 5. Segundo bloque

Cambia el baudrate al del otro bloque, reinicia, repite la lista de chequeo del paso 2 y ejecuta el paso 3 con **el mismo N**. Al terminar, repite el paso 4.

Criterio de cierre de la semana

- ☐ Dos bloques completos con el N decidido.
- ☐ resumen_rapido.py revisado en los dos; fallas menores al 5 % (o bloque repetido y documentado).
- ☐ Datos comprimidos y subidos a GitHub; copia en la laptop.
- ☐ Bitácora con fecha, hora, temperatura inicial e incidentes de cada bloque.

Cuando termines, mándame los dos resúmenes (`s10_resumen_estandar.txt` y `s10_resumen_rapido.txt`) y pasamos al análisis de la semana 11.